



ITES
RENÉ DESCARTES

CARRERA:
LICENCIATURA EN PROGRAMACIÓN
Y WEB MASTER

DOCENTE:
Mario Alonso Segovia Gutiérrez

MATERIA:
Proyecto de una Plataforma Virtual

ALUMNO:
CASTILLO PINZON CARLOS ALBERTO

FECHA:
20-07-2026

2. Introducción

La calidad del software es muy importante porque permite desarrollar sistemas que funcionen correctamente, sean fáciles de mantener y puedan mejorarse con el tiempo. Cuando un proyecto no sigue buenas prácticas de programación, es común que aparezcan errores, que el código sea difícil de entender y que el mantenimiento requiera más tiempo y esfuerzo.

En este trabajo se analizará el caso de estudio GymManager, un sistema para administrar un gimnasio que presenta diversos problemas en su desarrollo. A partir de este caso se identificarán las principales fallas del proyecto, se analizarán utilizando los conceptos de Clean Code y los principios SOLID, además de revisar la deuda técnica existente. También se propondrá un Producto Mínimo Viable (MVP) y una Definition of Done (DoD) con el objetivo de mostrar cómo la aplicación de buenas prácticas puede mejorar la calidad del software y facilitar su desarrollo y mantenimiento.

3. Identificación de problemas

Problema	¿Qué ocurre?	¿Por qué es una mala práctica?	Consecuencias
Clases demasiado grandes	Existen clases con más de 1,800 líneas de código.	Incumplen el principio de responsabilidad única y dificultan su comprensión.	Mayor dificultad para mantener y corregir errores.
Funciones extensas	Algunas funciones superan las 250 líneas.	Realizan demasiadas tareas dentro de un mismo método.	Código difícil de leer, probar y reutilizar.
Código duplicado	Se copia y pega código entre módulos.	Genera redundancia y aumenta el mantenimiento.	Los cambios deben realizarse en varios lugares, aumentando el riesgo de errores.
Variables poco descriptivas	Se utilizan nombres como <i>x</i> , <i>dato</i> o <i>temp</i> .	No expresan claramente el propósito de la variable.	El código resulta confuso para otros desarrolladores.
Múltiples responsabilidades	Un archivo valida datos, consulta la base de datos, genera reportes y envía correos.	Mezcla distintas responsabilidades en un solo componente.	Los cambios afectan varias funcionalidades al mismo tiempo.
Organización inconsistente	Cada desarrollador utiliza una estructura diferente.	No existe un estándar para organizar el proyecto.	Mayor dificultad para trabajar en equipo.
Ausencia de guía de estilos	No hay reglas para escribir código.	Se pierde consistencia en el desarrollo.	El mantenimiento se vuelve más complejo.
Comentarios innecesarios	Se explica cada línea mediante comentarios.	El código debería ser autoexplicativo.	Los comentarios quedan desactualizados y generan confusión.
No existen revisiones de código	Nunca se revisa el trabajo entre compañeros.	Los errores llegan directamente al proyecto principal.	Disminuye la calidad del software.
Modificaciones constantes	Cada nueva función requiere cambiar varias clases existentes.	El sistema presenta un alto acoplamiento.	Aumenta el riesgo de introducir nuevos errores.
Funcionalidades innecesarias	Se agregan características que el cliente nunca solicitó.	Se desperdician recursos en funciones sin valor.	Incrementan el tiempo y costo del desarrollo.
Documentación desactualizada	Nunca se actualiza la documentación.	La información deja de reflejar el estado real del sistema.	Dificulta el mantenimiento y la incorporación de nuevos desarrolladores.

4. Análisis de Clean Code

1. Clases demasiado grandes

Principio incumplido: Una clase debe tener una única responsabilidad.

Corrección: Dividir las clases en componentes más pequeños según su función.

Beneficio: El código será más sencillo de comprender, mantener y reutilizar.

2. Funciones muy extensas

Principio incumplido: Las funciones deben ser pequeñas y realizar una sola tarea.

Corrección: Extraer bloques de código en métodos independientes.

Beneficio: Mejora la legibilidad y facilita las pruebas unitarias.

3. Variables poco descriptivas

Principio incumplido: Utilizar nombres significativos.

Corrección: Reemplazar nombres genéricos por otros que describan su propósito.

Beneficio: El código resulta más claro y fácil de entender.

4. Código duplicado

Principio incumplido: Evitar la duplicación (DRY).

Corrección: Centralizar la lógica común en funciones o clases reutilizables.

Beneficio: Reduce errores y simplifica el mantenimiento.

5. Comentarios excesivos

Principio incumplido: El código debe ser autoexplicativo.

Corrección: Mejorar los nombres de variables, clases y métodos en lugar de documentar cada línea.

Beneficio: El código permanece actualizado y resulta más fácil de interpretar.

5. Aplicación de los principios SOLID

Principio de Responsabilidad Única (SRP)

En qué consiste: Cada clase debe tener una única responsabilidad.

Evidencia: Un mismo archivo valida información, consulta la base de datos, genera reportes y envía correos.

Cambio propuesto: Separar cada responsabilidad en clases independientes.

Beneficio: El mantenimiento será más sencillo y los cambios tendrán menor impacto.

Principio Abierto/Cerrado (OCP)

En qué consiste: Las clases deben poder extenderse sin modificar su código existente.

Evidencia: Cada nueva funcionalidad obliga a modificar varias clases.

Cambio propuesto: Utilizar interfaces y herencia para extender funcionalidades.

Beneficio: Se podrán agregar nuevas funciones sin afectar el código existente.

Principio de Inversión de Dependencias (DIP)

En qué consiste: Los módulos deben depender de abstracciones y no de implementaciones concretas.

Evidencia: Los módulos están fuertemente acoplados.

Cambio propuesto: Implementar interfaces e inyección de dependencias.

Beneficio: Facilita las pruebas y permite reemplazar componentes sin afectar al sistema.

Principio de Segregación de Interfaces (ISP)

En qué consiste: Cada componente debe depender únicamente de los métodos que necesita.

Evidencia: Las clases realizan múltiples funciones que no corresponden a una única interfaz.

Cambio propuesto: Crear interfaces específicas para cada módulo.

Beneficio: Reduce el acoplamiento y mejora la organización del proyecto.

6. Análisis de la deuda técnica

El proyecto presenta una deuda técnica considerable debido a decisiones tomadas para acelerar el desarrollo sin seguir buenas prácticas de ingeniería de software.

Las principales fuentes de deuda técnica son el código duplicado, las clases excesivamente grandes, la falta de documentación, la ausencia de revisiones de código, la incorporación de funcionalidades innecesarias y la inexistencia de criterios para considerar terminada una tarea.

Esta deuda provocará un aumento constante en los tiempos de mantenimiento, ya que cada cambio requerirá modificar múltiples componentes y aumentará la probabilidad de introducir nuevos errores. Además, el costo del desarrollo crecerá debido a la necesidad de invertir más tiempo en corregir fallos que en implementar nuevas funcionalidades.

Para disminuir la deuda técnica se recomienda:

- Refactorizar las clases y funciones más grandes.
- Eliminar el código duplicado.
- Implementar revisiones de código.
- Establecer una guía de estilos.
- Mantener la documentación actualizada.
- Incorporar pruebas automatizadas.
- Definir criterios claros para aprobar funcionalidades antes de integrarlas al proyecto.

Con estas acciones el sistema será más estable, mantenible y escalable.

7. Definición del MVP

Funcionalidades incluidas en el MVP

- Gestión de clientes.
- Registro de pagos.
- Control de asistencia.
- Reservación de clases.
- Reportes básicos de ingresos.

Justificación

Estas funcionalidades representan el funcionamiento esencial del gimnasio. Permiten registrar clientes, controlar pagos, administrar la asistencia y conocer los ingresos del negocio. Son las características de mayor valor para el cliente y las que hacen posible operar el sistema desde su primera versión.

Funcionalidades para versiones futuras

- Chat interno entre entrenadores.
- Integración con relojes inteligentes.
- Sistema de recompensas.
- Publicación automática en redes sociales.
- Tienda en línea de suplementos.

Justificación

Aunque estas funciones agregan valor, no son indispensables para el funcionamiento inicial del sistema. Su desarrollo requiere mayor tiempo, recursos e integración con servicios externos, por lo que pueden implementarse una vez que el sistema principal sea estable.

8. Definition of Done (DoD)

Una funcionalidad se considerará terminada únicamente cuando cumpla con los siguientes criterios:

1. Cumple completamente los requerimientos solicitados por el cliente.
2. El código sigue las normas de Clean Code.
3. Se respetan los principios SOLID cuando aplican.
4. No existe código duplicado.
5. Las pruebas unitarias y funcionales fueron ejecutadas correctamente.
6. La funcionalidad fue revisada y aprobada mediante revisión de código.
7. No presenta errores críticos conocidos.
8. La documentación técnica fue actualizada.
9. La funcionalidad fue integrada correctamente al repositorio principal.
10. El Product Owner o responsable del proyecto validó su correcto funcionamiento.

9. Conclusión

El caso de GymManager demuestra que no aplicar buenas prácticas de programación puede generar muchos problemas durante el desarrollo de un sistema. Esto provoca que el código sea más difícil de entender, aumenten los errores y el mantenimiento requiera más tiempo y dinero.

Conceptos como Clean Code, los principios SOLID, la deuda técnica y el MVP ayudan a desarrollar software más organizado, fácil de mantener y preparado para futuras mejoras. Aplicar estas prácticas también facilita el trabajo en equipo y mejora la calidad del producto.

En mi proyecto pondré en práctica estos conocimientos escribiendo código más claro, organizando mejor las funciones, evitando repetir código y documentando los cambios importantes. De esta forma podré desarrollar un sistema más ordenado, fácil de mantener y que cumpla con las necesidades del usuario.